

Selenium IDE Automation Lab

Practical Solutions for Parameterization, Verification & Test Suites

TASK 1 Flipkart Homepage Verification

1. Install the **Selenium IDE** browser extension (Chrome / Firefox) and launch it from your browser toolbar.
2. Select **Record a new test in a new project**, name the project `Flipkart_Tests`, and set the base URL to `https://www.flipkart.com`.
3. Click **Start Recording**, navigate to Flipkart, and dismiss any initial login modal.
4. Right-click the page/logo area → **Selenium IDE** → **Verify** → **Text**:
 - **Command:** `verifyText`
 - **Target:** `css=body`
 - **Value:** `Flipkart`
5. Stop recording and save the test case as `Verify_Flipkart_Title`.

TASK 2 Amazon India Search Verification

1. Create a new test named `Amazon_Search` and start recording at `https://www.amazon.in`.
2. Click the search bar, type `Bluetooth headphones`, and click the search button.
3. Once search results render, verify the result header contains the target term:
 - **Command:** `verifyText` (or `assertText`)
 - **Target:** `xpath=//span[contains(text(), 'headphones')]`
 - **Value:** `headphones`
4. Stop recording and run the test to ensure execution succeeds.

TASK 3 Demo App Login Verification

Record a login test case at `https://practicetestautomation.com/practice-test-login/` using the command structure below:

COMMAND	TARGET	VALUE
open	/practice-test-login/	(blank)
type	id=username	student
type	id=password	Password123
click	id=submit	(blank)
verifyText	css=.has-text-align-center	Logged In Successfully

TASK 4 Parameterized Search Test on Myntra

Use the `store` command to set a search variable dynamically for multiple searches:

```
1. Command: store      | Target: shoes      | Value: searchTerm
2. Command: open       | Target: https://www.myntra.com
3. Command: type       | Target: css=.desktop-searchBar | Value: ${searchTerm}
4. Command: sendKeys   | Target: css=.desktop-searchBar | Value: ${KEY_ENTER}
5. Command: verifyText | Target: css=.title-title      | Value: ${searchTerm}

6. Command: store      | Target: t-shirts | Value: searchTerm
7. Command: open       | Target: https://www.myntra.com
8. Command: type       | Target: css=.desktop-searchBar | Value: ${searchTerm}
9. Command: sendKeys   | Target: css=.desktop-searchBar | Value: ${KEY_ENTER}
10. Command: verifyText | Target: css=.title-title      | Value: ${searchTerm}
```

TASK 5 Zomato Test Suite & Script Export Benefits

1. **Create Tests:** Record `Zomato_Homepage_Load` (verifying title) and `Zomato_Search_Pizza` (searching for 'pizza').
2. **Build Suite:** Open the **Test Suites** panel, create `Zomato_Suite`, and add both tests to it.
3. **Export Script:** Right-click `Zomato_Suite` → **Export** → select preferred target (e.g., Python PyTest or Java JUnit).

```
# EXPORT BENEFIT COMMENT FOR QA TEAMS:  
# Exporting Selenium IDE tests to code (such as Python PyTest or Java JUnit) allows QA  
# teams  
# to transition simple record-and-playback scripts into maintainable, code-based  
# automation frameworks.  
# This enables integration with CI/CD pipelines (e.g., GitHub Actions, Jenkins), parallel  
# test execution,  
# dynamic cross-browser grid testing (Selenium Grid/BrowserStack), and custom reporting  
# libraries.
```